Distributed Automation Platform for Mars IoT Simulator

•••

Laboratory of advanced programming, Sapienza Università di Roma
Members: Gabriele Bruni, Luca Di Carlo

Mission: Welcome to Mars. Please Don't Die.

The goal of this mission is to implement a distributed automation system capable of ingesting data from different sensors, normalize them into a Unified Event Schema and use them to enforce simple automation rules. The system should also provide a live dashboard for real-time monitoring.

The system has been implemented using the following frameworks:

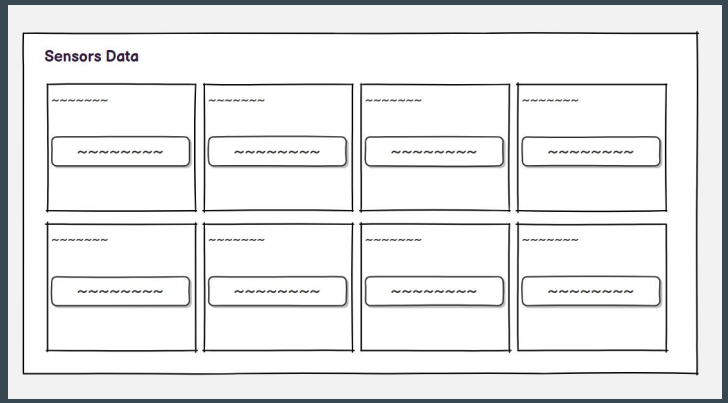
- Python 3 & FastAPI, for the backend/frontend logic, asynchronous worker scripts, and REST/SSE APIs;
- RabbitMQ, as the message broker ensuring event-driven decoupling between ingestion and processing;
- MariaDB, as the relational database for the persistent storage of automation rules;
- HTML5, Vanilla JS & Bootstrap: for a lightweight and reactive frontend dashboard.
- Docker: for the containerization and orchestration of the microservices ecosystem.

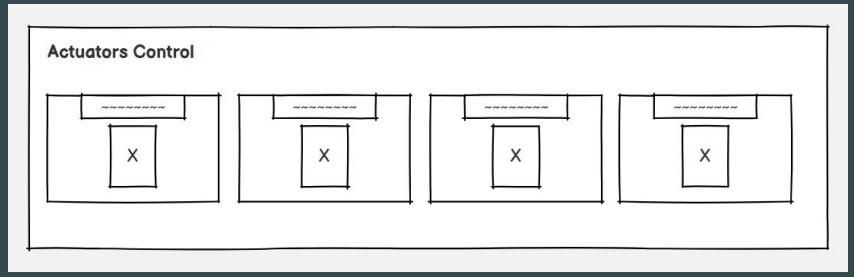
User stories and requirements

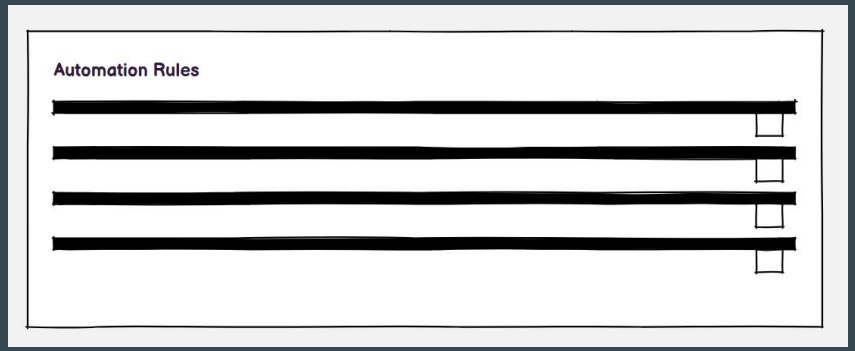
Being a group of 2, we created 10 user stories:

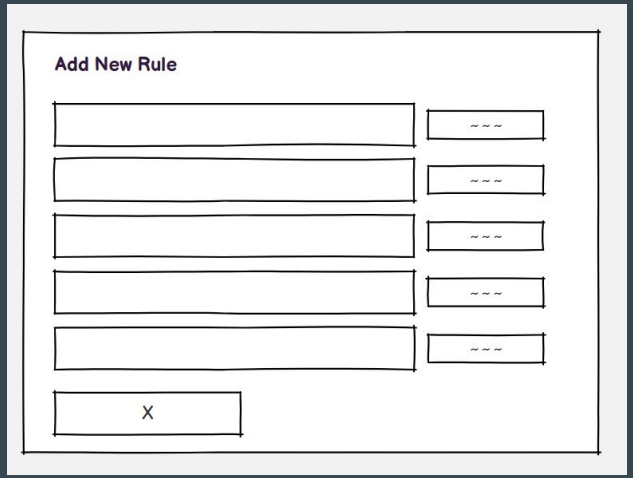
1. As an Operator, I want to see a real-time dashboard with the latest sensor values so that I can monitor the habitat conditions.
2. As an Operator, I want to view the current status of all actuators so that I know what equipment is currently operating.
3. As an Operator, I want to manually switch actuators ON or OFF from the dashboard so that I can intervene when necessary.
4. As an Operator, I want to create an automation rule from the dashboard so that the system can react automatically to sensor changes.
5. As an Operator, I want to view the list of active automation rules so that I know what behaviors are currently configured.
6. As an Operator, I want to delete an existing automation rule so that I can remove obsolete behaviors.
7. As the System, I want to poll data from REST sensors periodically so that the latest environmental data is seamlessly fetched.
8. As the System, I want to normalize the varying payloads from different REST sensors into a unified event format so that downstream services can process them uniformly.
9. As the System, I want to evaluate incoming sensor events dynamically against persisted rules so that the system safely triggers actuators when environmental conditions change.
10. As the System, I want to persist the created automation rules in an internal database so that they survive system restarts.

LoFi Mockups







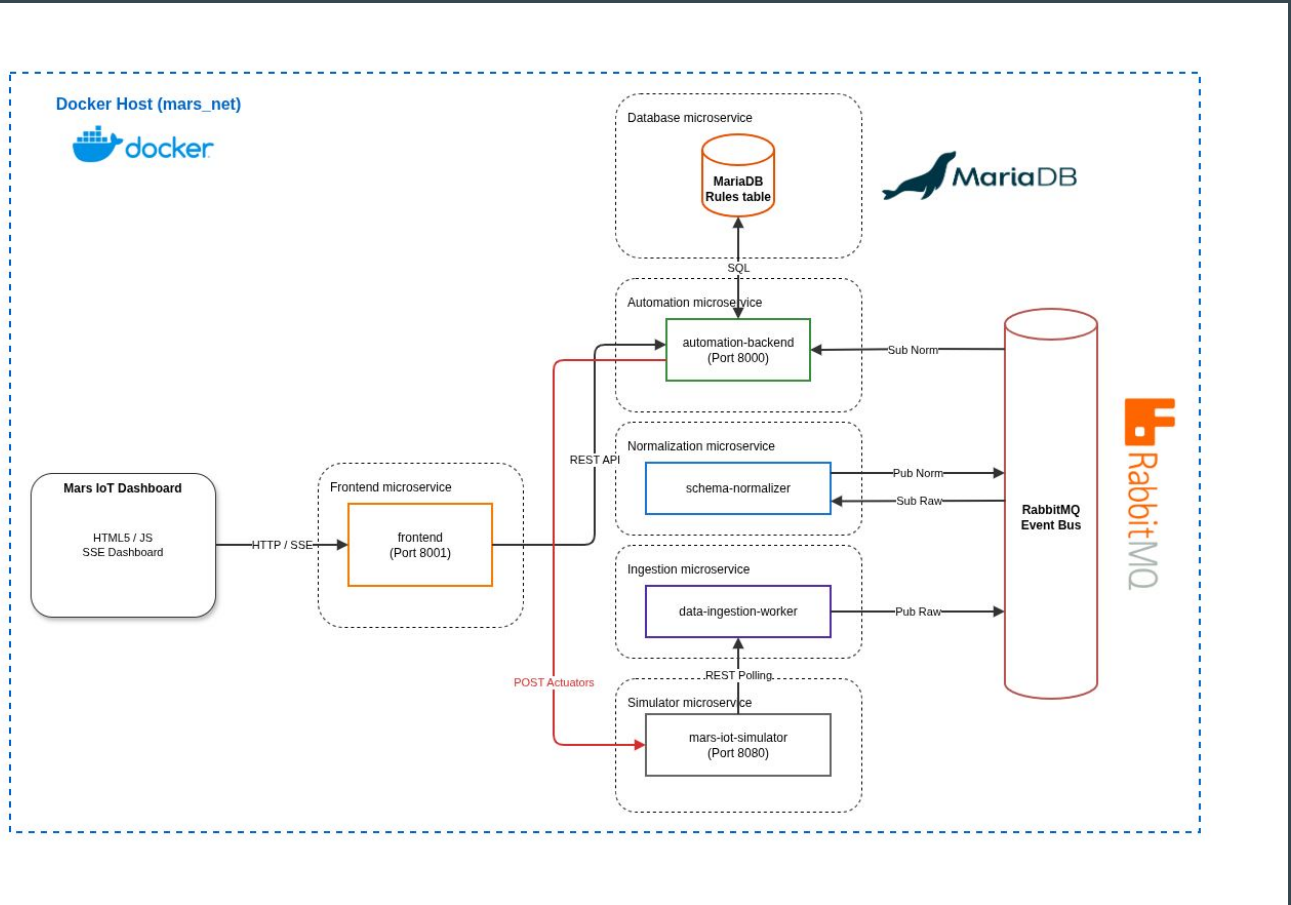


Microservices Architecture

The entire system is containerized using Docker to ensure portability and isolation.

Our system is composed of several decoupled modules:

- Simulator: Martian IoT environment (black-box);
- Ingestion: collects raw data from the simulator;
- Normalization: responsible for data standardisation through the UES;
- Message Broker: for asynchronous communication;
- Automation Engine: enforces automation logic;
- Database: for automation rules persistence;
- Frontend: the operator dashboard.



Unified Event Schema

The secret ingredient of this system is how data normalization is performed in order to reconstruct the information brought by different sensors in a common standard:

```
{
  "sensor_id": "string",
  "captured_at": "string (ISO 8601 date-time)",
  "metric": "string",
  "value": "number",
  "unit": "string",
  "status": "string (ok | warning)"
}
```

Our UES aims at processing each event as a couple “sensor-metric”, as sensors with multiple metrics have to be broken down into multiple events. This allows us to generalize multi-metric sensors into scalar sensors.

Unified Event Schema (visual idea)

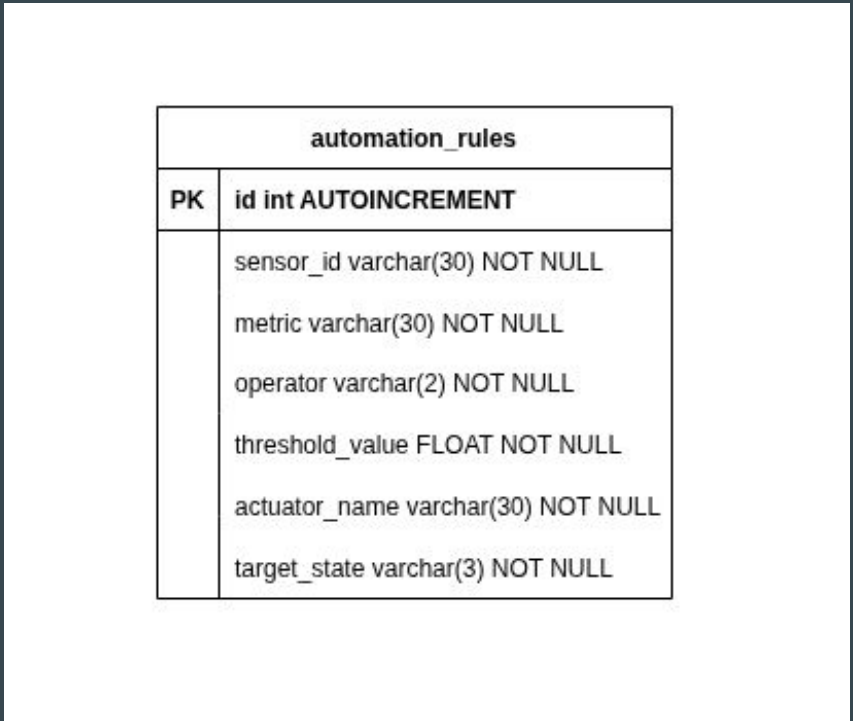
```
### `rest.scalar.v1` (greenhouse_temperature, entrance_humidity, co2_hall, corridor_pressure)
```json
{
  "type": "object",
  "required": ["sensor_id", "captured_at", "metric", "value", "unit", "status"],
  "properties": {
    "sensor_id": { "type": "string" },
    "captured_at": { "type": "string", "format": "date-time" },
    "metric": { "type": "string" },
    "value": { "type": "number" },
    "unit": { "type": "string" },
    "status": { "type": "string", "enum": ["ok", "warning"] }
  }
}
```
```

```
### `rest.chemistry.v1` (hydroponic_ph, air_quality_voc)
```json
{
  "type": "object",
  "required": ["sensor_id", "captured_at", "measurements", "status"],
  "properties": {
    "sensor_id": { "type": "string" },
    "captured_at": { "type": "string", "format": "date-time" },
    "measurements": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["metric", "value", "unit"],
        "properties": {
          "metric": { "type": "string" },
          "value": { "type": "number" },
          "unit": { "type": "string" }
        }
      }
    },
    "status": { "type": "string", "enum": ["ok", "warning"] }
  }
}
```
```

```
### `rest.particulate.v1` (air_quality_pm25)
```json
{
  "type": "object",
  "required": ["sensor_id", "captured_at", "pm1_ug_m3", "pm25_ug_m3", "pm10_ug_m3", "status"],
  "properties": {
    "sensor_id": { "type": "string" },
    "captured_at": { "type": "string", "format": "date-time" },
    "pm1_ug_m3": { "type": "number" },
    "pm25_ug_m3": { "type": "number" },
    "pm10_ug_m3": { "type": "number" },
    "status": { "type": "string", "enum": ["ok", "warning"] }
  }
}
```
```

```
{
  "sensor_id": "string",
  "captured_at": "string (ISO 8601 date-time)",
  "metric": "string",
  "value": "number",
  "unit": "string",
  "status": "string (ok | warning)"
}
```

Rule Model & Logical Structure



automation_rules	
PK	id int AUTOINCREMENT
	sensor_id varchar(30) NOT NULL
	metric varchar(30) NOT NULL
	operator varchar(2) NOT NULL
	threshold_value FLOAT NOT NULL
	actuator_name varchar(30) NOT NULL
	target_state varchar(3) NOT NULL

Logical structure:

- Visual representation:
  IF <Sensor> [<Metric>] <Operator> <Threshold> THEN SET <Actuator> to <State>;
- Key blocks: Condition -> Action.

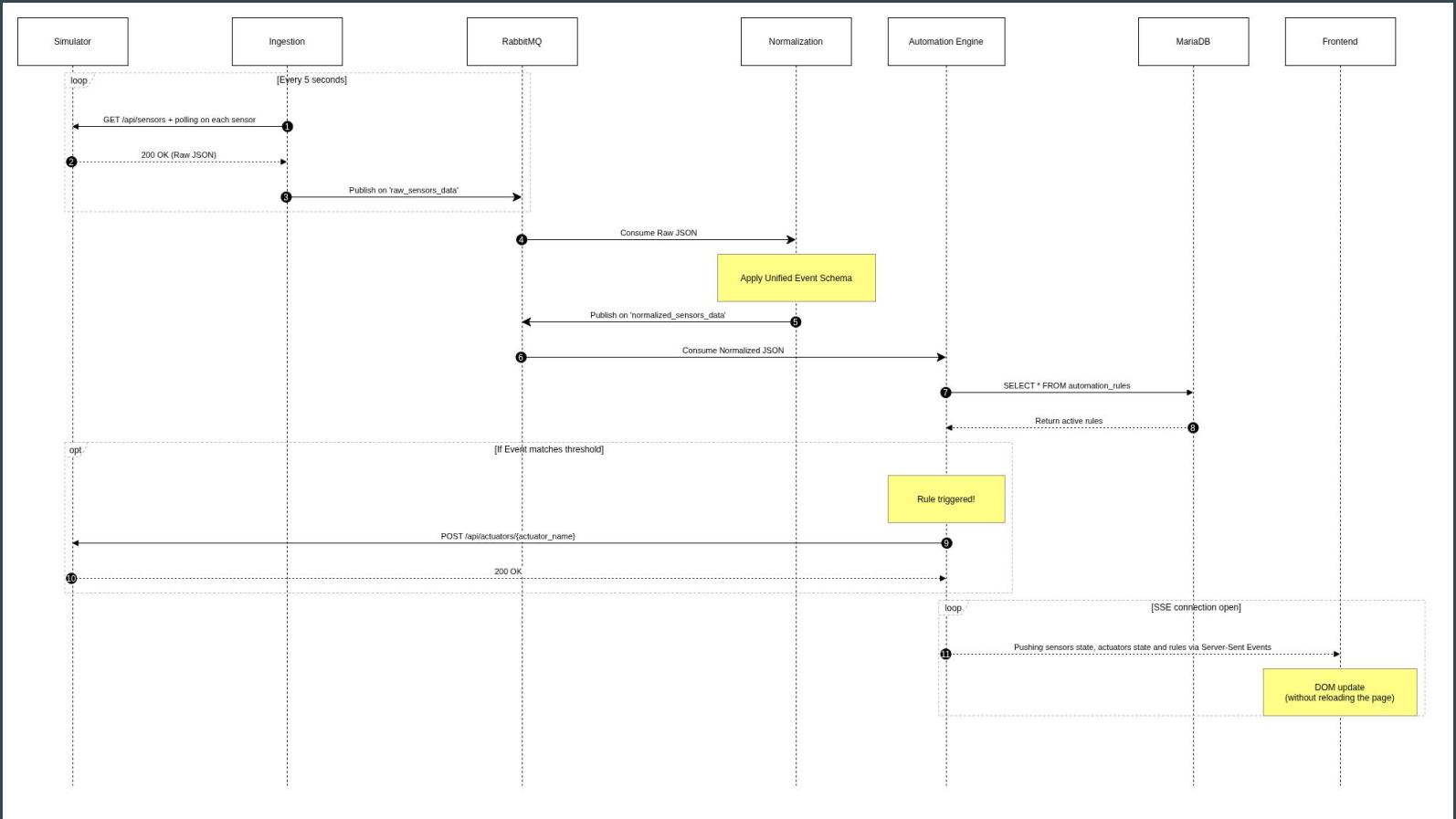
Database & persistence:

- Securely store in automation_rules table;
- Ensures automation behaviors are fully restored upon system crashes or service restarts.

System integration:

- Rules are continuously applied on the normalized data sensors;
- Active interaction with actuators APIs (e.g. turning ON the cooling_fan).

Sequence diagram



Live Demo & Q&A

The following demonstration showcases a complete docker-compose deployment, illustrating data ingestion from simulated sensors, rule evaluation in the automation engine, and real-time dashboard updates via SSE.

Ready to execute: docker compose up